

SIMATS ENGINEERING
ASSESSMENT TOOL 1
COURSE NAME: Artificial Intelligence
COURSE CODE: CSA17

Code of Conduct:

I, **B.Vishnu / 192512410** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

ANNEXURE A
INTEGRATED EXPERIMENTS - SOLUTIONS

Experiment 1: Decision Tree Classification

(a) Load Dataset, Pre-process, Split, and Display

We utilize the standard Iris dataset. Since it is clean, no missing value handling is strictly required, but we verify it. We encode the target labels (Species) and split the data 80/20.

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# Load dataset
iris = load_iris()
df = pd.DataFrame(data=iris.data, columns=iris.feature_names)
df['species'] = iris.target

# Split into train and test sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(
    df.iloc[:, :-1], df['species'], test_size=0.2, random_state=42
)

print("First 5 rows:")
print(df.head())
print("\nData Types:")
print(df.dtypes)
```

First 5 rows:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
species				
0	5.1	3.5	1.4	
0.2	0			
1	4.9	3.0	1.4	
0.2	0			
2	4.7	3.2	1.3	
0.2	0			
3	4.6	3.1	1.5	
0.2	0			
4	5.0	3.6	1.4	
0.2	0			

Data Types:

sepal length (cm)	float64
sepal width (cm)	float64
petal length (cm)	float64
petal width (cm)	float64
species	int64
dtype:	object

(b) Build Decision Tree Model and Identify Root Node

We build the Decision Tree using the Gini Index as the splitting criterion. The root node is selected based on the feature that provides the lowest Gini impurity (or highest information gain).

```
from sklearn.tree import DecisionTreeClassifier, export_text

# Initialize and train the model using Gini index
dt_classifier = DecisionTreeClassifier(criterion='gini', random_state=42)
dt_classifier.fit(X_train, y_train)

# Print tree structure
tree_rules = export_text(dt_classifier, feature_names=list(X_train.columns))
print(tree_rules)
```

```
|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) > 2.45
|   |--- petal width (cm) <= 1.75
|       |--- petal length (cm) <= 4.95
|           |--- class: 1
... (truncated for brevity)
```

Root Node Justification: The root node chosen by the algorithm is **petal length (cm) ≤ 2.45** . This feature is selected because it perfectly separates the 'Setosa' class (class 0) from the 'Versicolor' and 'Virginica' classes. By splitting here, the algorithm maximizes the decrease in Gini impurity, yielding one pure leaf node immediately.

(c) Evaluate the Model and Misclassified Instances

```
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

y_pred = dt_classifier.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
```

```
Accuracy: 1.0
Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
Classification Report:
              precision    recall  f1-score   support

     0           1.00        1.00        1.00         10
     1           1.00        1.00        1.00          9
     2           1.00        1.00        1.00         11
 accuracy                   1.00                   30
```

Performance Comment: On the standard 80/20 split with random state 42, the Decision Tree achieves 100% accuracy. The model perfectly generalizes to the test set due to the distinct clustering of the Iris species.

Misclassified Instances Identification: Because our standard split achieved 100%, we theoretically analyze where misclassifications typically occur in this dataset. When varying the random seed (e.g., `random_state=0`), overlapping data points occur between *Versicolor* (1) and *Virginica* (2).

Example of typical misclassifications:

1. A flower with `petal length = 4.8` and `petal width = 1.8` (Actual: *Versicolor*, Predicted: *Virginica*).
2. A flower with `petal length = 5.1` and `petal width = 1.5` (Actual: *Virginica*, Predicted: *Versicolor*).

These occur because the boundary between *Versicolor* and *Virginica* is non-linear and slightly overlapping in the petal dimension space.

Experiment 2: Reinforcement Learning - Q-Learning

(a) Define Environment and Hyperparameters

We define a 4x4 Grid World (16 states: 0 to 15).

- **States:** 0 to 15 (0 is top-left, 15 is bottom-right).
- **Actions:** 0: Up, 1: Down, 2: Left, 3: Right.
- **Rewards:** Goal (State 15) = +10, Obstacles (States 5, 10) = -5, Default step = -1.
- **Hyperparameters:** Learning Rate (α) = 0.1, Discount Factor (γ) = 0.9, Exploration Rate (ϵ) = 0.1.

The Q-Table is initialized as a 16x4 matrix of zeros.

(b) Run Q-Learning Algorithm and Record Q-values

During training, the agent updates its Q-values using the Bellman equation:

$$Q(s,a) \leftarrow Q(s,a) + \alpha * [R + \gamma * \max_{a'}(Q(s',a')) - Q(s,a)]$$

Below is a simulated tracking of the Q-table evolution for State 0 (Start) and State 14 (Adjacent to goal).

Episode	State 0 (Start) Q-Values [U, D, L, R]	State 14 (Adj Goal) Q-Values [U, D, L, R]
1	[0.0, 0.0, 0.0, 0.0]	[0.0, 0.0, 0.0, 1.0] (<i>Found goal once</i>)
50	[-0.5, 0.2, -0.5, 0.3]	[-0.2, -0.5, 0.1, 8.1] (<i>Propagating rewards</i>)
100	[-1.2, 1.5, -1.2, 1.6]	[-0.8, -1.0, 0.5, 9.9] (<i>Converging</i>)

As episodes increase, the +10 reward from State 15 propagates backwards. State 14 heavily favors the 'Right' action (value approaches ~10), and State 0 begins favoring paths leading away from obstacles and towards the goal.

(c) Extract Learned Policy and Convergence Behaviour

```
# Extracted Optimal Policy (Action with max Q-value per state)
Policy Grid:
[ Right, Right, Down,  Down ]
[ Down,  OBST,  Down,  Down ]
[ Right, Right, OBST,  Down ]
[ Right, Right, Right, GOAL ]
```

Convergence Behaviour Justification:

If we plot the cumulative reward per episode, the graph would start with highly negative values (e.g., -50 to -100) during the initial episodes. This is because the agent explores randomly, frequently hitting the -5 obstacles and accumulating -1 step penalties without finding the goal efficiently.

Around Episode 40-60, the cumulative reward sharply increases and plateaus at a positive constant (e.g., +4). This plateau indicates **convergence**. The agent has optimized its policy, consistently avoiding obstacles and taking the shortest path (6 steps: -6 reward + 10 goal reward = +4 net reward). The epsilon value ($\epsilon=0.1$) ensures a slight noise remains, but the greedy policy is fully established.